Code

Stats 20, F23 – Homework 2

Aashna Sibal

10/17/2023

Question 1

```
mixed1 <- c(TRUE, FALSE, FALSE, TRUE, 4, 0, 3)
mixed1
```

```
## [1] 1 0 0 1 4 0 3
```

```
mixed2 <- c(TRUE, FALSE, FALSE, TRUE, 4, 0, "3")
mixed2
```

```
## [1] "TRUE"  "FALSE" "FALSE" "TRUE"  "4"     "0"     "3"
```

```
mixed3 <- c(c(TRUE, FALSE, FALSE, TRUE, 4, 0), "3")
mixed3
```

```
## [1] "1" "0" "0" "1" "4" "0" "3"
```

(a)

Mixed2 is a vector with one character, and since characters are the highest in the mode hierarchy, mixed2 produces all characters in the output. In mixed3, there is a vector inside a vector with a character, and the inner one's highest mode is numeric, so all the logical values get turned into numbers. Then the outer vector has a character so all the numerics become characters, producing an output of characters that are all numbers.

(b)

```
c(TRUE, FALSE, c(FALSE, TRUE, 4, 0), "3")
```

```
## [1] "TRUE"  "FALSE" "0"     "1"     "4"     "0"     "3"
```

Question 2

(a)

```
as.numeric(mixed2)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] NA NA NA NA  4  0  3
```

```
as.numeric(mixed3)
```

```
## [1] 1 0 0 1 4 0 3
```

In mixed2, the values are all characters, as we saw in question one. Therefore, R does not know what to convert them to as numerics, because the logical factor is not present. It just says the words TRUE and FALSE rather than being of the logical type, so R cannot recognize them. With mixed 3, R sees all number characters can easily convert them into numeric types.

(b)

```
as.logical(mixed2)
```

```
## [1]  TRUE FALSE FALSE  TRUE    NA    NA    NA
```

```
as.logical(mixed3)
```

```
## [1] NA NA NA NA NA NA NA
```

In mixed2, R can tell from the characters “TRUE” or “FALSE” what the logical values can be. Since there are numbers, R designates those NA values. For mixed3, there is no logical value tied to these all numeric and then all character values, so R does not know how to make them logical types.

(c)

```
as.logical(as.numeric(mixed3))
```

```
## [1]  TRUE FALSE FALSE  TRUE  TRUE FALSE  TRUE
```

Question 3

(a)

```
seq(1,25)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
22 23 24 25
```

```
1:25
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
22 23 24 25
```

```
seq_len(25)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
22 23 24 25
```

```
seq_along(seq(1,25))
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
22 23 24 25
```

(b)

```
seq(8, 2, by = -.5)
```

```
##  [1] 8.0 7.5 7.0 6.5 6.0 5.5 5.0 4.5 4.0 3.5 3.0 2.5 2.0
```

```
rep(seq(-1, 5, length = 4),5)
```

```
##  [1] -1  1  3  5 -1  1  3  5 -1  1  3  5 -1  1  3  5 -1  1  3  5
```

```
rep(seq(5, 1, by = -1),5)
```

```
##  [1] 5 4 3 2 1 5 4 3 2 1 5 4 3 2 1 5 4 3 2 1 5 4 3 2 1
```

```
rep(seq(5, 1, by = -1),(rep(5,5)))
```

```
##  [1] 5 5 5 5 5 4 4 4 4 4 3 3 3 3 3 2 2 2 2 2 1 1 1 1 1
```

```
rep(seq(10, 6, by = -1),(seq(1,5)))
```

```
##  [1] 10  9  9  8  8  8  7  7  7  7  6  6  6  6  6
```

(c)

This cannot be created with only seq() and rep() since it is not a consistent repetition of the same sequence or numbers. Each chunk is a different set of five numbers and unlike the patterns in b, there is a decrement involved in the WHOLE chunk in each step.

```
rep(seq(10,6),5) - rep(seq(0, 4),(rep(5,5)))
```

```
##  [1] 10  9  8  7  6  9  8  7  6  5  8  7  6  5  4  7  6  5  4  3
6  5  4  3  2
```

Question 4

```
k <- 0:100
sin(k * pi)
```

```
##   [1]  0.000000e+00  1.224647e-16 -2.449294e-16  3.673940e-16
-4.898587e-16
##   [6]  6.123234e-16 -7.347881e-16  8.572528e-16 -9.797174e-16
1.102182e-15
##  [11] -1.224647e-15  4.899825e-15 -1.469576e-15 -1.960673e-15
-1.714506e-15
##  [16]  5.389684e-15 -1.959435e-15 -1.470814e-15 -2.204364e-15
5.879543e-15
##  [21] -2.449294e-15 -9.809554e-16 -9.799650e-15  6.369401e-15
-2.939152e-15
##  [26] -4.910967e-16  3.921346e-15  6.859260e-15 -3.429011e-15
-1.237961e-18
##  [31] -1.077937e-14  7.349119e-15 -3.918870e-15  4.886208e-16
2.941628e-15
##  [36]  7.838977e-15 -4.408728e-15  9.784795e-16 -1.175909e-14
8.328836e-15
##  [41] -4.898587e-15  1.567919e-14  1.961911e-15  8.818695e-15
-1.959930e-14
##  [46]  1.958197e-15 -1.273880e-14 -4.902301e-15 -5.878305e-15
1.665891e-14
##  [51]  9.821934e-16  9.798412e-15  7.842691e-15  2.937914e-15
-1.371852e-14
##  [56] -3.922584e-15 -6.858022e-15  1.763863e-14  2.475923e-18
1.077813e-14
##  [61] -2.155874e-14  3.917632e-15 -1.469824e-14 -2.942866e-15
-7.837740e-15
##  [66]  1.861835e-14 -9.772415e-16  1.175785e-14  5.883256e-15
4.897349e-15
##  [71] -1.567795e-14 -1.963149e-15 -8.817457e-15  1.959806e-14
-1.956959e-15
##  [76]  1.273756e-14 -2.351817e-14  5.877067e-15 -1.665767e-14
-9.834313e-16
##  [81] -9.797174e-15  2.057778e-14 -3.135839e-14 -1.470443e-14
3.923822e-15
##  [86]  6.856784e-15 -1.763739e-14  2.841800e-14 -3.919860e-14
-6.864212e-15
##  [91] -3.916394e-15  1.469700e-14 -2.547761e-14  3.625821e-14
9.804602e-15
##  [96]  9.760036e-16 -1.175661e-14  2.253721e-14 -3.331782e-14
-1.274499e-14
## [101]  1.964387e-15
```

No, mathematically I would expect to see mainly 0s and 1s, since that is what the since of multiples of pi is.

Question 5

(a)

```
my_skew <- function(x) {

  (sqrt(length(x)) * sum((x - mean(x))^3)) / (sum((x - mean(x))^2)) ^
(3/2)
}
```

(b)

```
running_times <- c(51, 40, 57, 34, 47, 50, 50, 56, 41, 38)
my_skew(running_times)
```

```
## [1] -0.1615082
```

This value means that the running times distribution is slightly skewed to the left, or has a tail on the left end.

(c)

```
my_skew_one <- function(x) {
  n <- length(x)
  x_bar <- mean(x)

  (sqrt(n) * (sum(x^3) - 3*x_bar*sum(x^2) + 2*n*(x_bar^3))) /
((sum(x^2) - n*(x_bar^2))^(3/2))
}
```

(d)

```
new <- running_times*1e10
my_skew_one(new)
```

```
## [1] -0.1615082
```

I got the same exact value in b and d. I think this is correct because all we did was algebraically manipulate the equation, in the end it is still calculating the same value. Also, we are multiplying ALL the values by the same number, so the skew is the same.

(e)

```
my_skew(running_times) - my_skew_one(running_times)
```

```
## [1] -4.15501e-14
```

This happens due to rounding errors. When there are very large values and one-pass functions, the inner terms being subtracted from each other lose a lot of precision. Because of this loss of precision, there are minimal tiny differences between the two-pass function result and that of the one-pas function.

Question 6

(a)

```
my_length <- function(x) {
  max(seq_along(x),0)
}
```

(b)

```
my_length(numeric(0))
```

```
## [1] 0
```

```
my_length(seq(1, 99, by = 2))
```

```
## [1] 50
```

```
my_length(c("waffles", "friends", "work"))
```

```
## [1] 3
```

Question 7

(a)

```
my_rev <- function(x) {
  x[(length(x):1) * (length(x) >= 1)]
}
```

(b)

```
my_rev(numeric(0))
```

```
## numeric(0)
```

```
my_rev(seq(1, 99, by = 2))
```

```
##  [1] 99 97 95 93 91 89 87 85 83 81 79 77 75 73 71 69 67 65 63 61 59
57 55 53 51
## [26] 49 47 45 43 41 39 37 35 33 31 29 27 25 23 21 19 17 15 13 11
9  7  5  3  1
```

```
my_rev(c("waffles", "friends", "work"))
```

```
## [1] "work"    "friends" "waffles"
```

Question 8

(a)

```
front <- function(x, n) {
  x[1:min(length(x),n) * (length(x) >= 1)]
}
front(numeric(0),6)
```

```
## numeric(0)
```

```
front(seq(1, 99, by = 2),6)
```

```
## [1]  1  3  5  7  9 11
```

```
front(seq(1,5),6)
```

```
## [1] 1 2 3 4 5
```

```
front(c("waffles", "friends", "work"),6)
```

```
## [1] "waffles" "friends" "work"
```

(b)

```
back <- function(x, n) {
  x[length(x):max(1,(length(x) - n + 1)) * (length(x) >= 1)]
}

back(numeric(0),6)
```

```
## numeric(0)
```

```
back(seq(1, 99, by = 2),6)
```

```
## [1] 99 97 95 93 91 89
```

```
back(c("waffles", "friends", "work"),6)
```

```
## [1] "work"    "friends" "waffles"
```

(c)

```
insert_at <- function(x, values, at) {
  beginning <- front(x,at-1)
  reverse_end <- back(x,length(x) - at +1)
  ending <- back(reverse_end, length(reverse_end))
  new_vec <- c(beginning, values, ending)
  new_vec
}
```

(d)

```
insert_at(1:5, c(0, 0, 0), at = 1)
```

```
## [1] 1 0 0 0 1 2 3 4 5
```

```
insert_at(1:5, c(0, 0, 0), at = 3)
```

```
## [1] 1 2 0 0 0 3 4 5
```

```
insert_at(1:5, c(0, 0, 0), at = 5)
```

```
## [1] 1 2 3 4 0 0 0 5
```

```
insert_at(1:5, c(0, 0, 0), at = 6)
```

```
##  [1]  1  2  3  4  5  0  0  0 NA  5
```